

LAPORAN SISTEM OPERASI

TUGAS 1 – 4



Dosen Pembimbing:

Triawan Adi Cahyanto M.Kom

Dibuat oleh:

Krisna Aji Pratama [2410651048]

TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi komputasi modern tidak lepas dari peran vital sistem operasi sebagai pengelola sumber daya perangkat keras dan penyedia layanan bagi aplikasi. Pemahaman mendalam mengenai konsep-konsep fundamental sistem operasi seperti manajemen proses, komunikasi antar proses, sistem file, dan antarmuka pengguna menjadi kompetensi esensial bagi calon ahli teknik informatika. Implementasi praktik langsung melalui pemrograman sistem menggunakan bahasa C memungkinkan mahasiswa untuk mengalami secara nyata mekanisme internal sistem operasi, yang seringkali abstrak jika hanya dipelajari secara teoritis. Tugas ini dirancang untuk membangun pemahaman holistik melalui pengembangan empat program utilitas yang mereplikasi fungsi inti sistem operasi, sekaligus melatih kemampuan pemecahan masalah dan debugging dalam lingkungan pemrograman sistem yang kompleks dan dekat dengan kernel.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam tugas ini adalah:

1. Bagaimana mengimplementasikan manajemen proses yang dapat membuat, mengelola, dan memfasilitasi komunikasi antara beberapa proses *child* dan *parent* secara efisien?
2. Bagaimana membangun sebuah mekanisme pemantauan sistem file yang dapat mendeteksi dan mencatat perubahan pada direktori tertentu secara *real-time*?
3. Bagaimana merancang dan mengembangkan sebuah *custom shell* yang mampu mengeksekusi perintah, *piping*, *redirection*, serta memiliki fitur *built-in command* dan *history*?

1.3 Tujuan

Adapun tujuan dari pelaksanaan tugas ini adalah:

1. Memahami dan mempraktikkan penggunaan *system call* seperti `fork()`, `waitpid()`, `pipe()`, dan `exec()` untuk mengelola proses dan komunikasi antar proses.
2. Membangun keterampilan dalam memanfaatkan API sistem seperti `inotify` untuk memantau aktivitas sistem file dan membuat program utilitas yang efektif.
3. Mengembangkan sebuah *command-line interpreter (shell)* sederhana yang mendukung eksekusi perintah dasar, *background process*, *I/O redirection*, dan *piping* untuk memperdalam pemahaman tentang antarmuka antara pengguna dan sistem operasi.

BAB II

PEMBAHASAN

2.1 Tugas Process Manager

2.1.1 Program Process_Manager

Program ini mengimplementasikan process manager menggunakan system call `fork()` untuk membuat tiga child process. Setiap child process menjalankan tugas berbeda: menghitung faktorial, mencari bilangan prima, dan membalik string. Parent process menggunakan `waitpid()` untuk menunggu setiap child selesai, `pipe()` untuk komunikasi antar process, dan `gettimeofday()` untuk mengukur waktu eksekusi. Program juga menangani sinyal `SIGINT` untuk terminasi yang graceful sesuai requirement NIM genap. Hasil komputasi setiap child dikirim ke parent melalui pipe dan ditampilkan bersamaan dengan status exit dan waktu eksekusi.

```
asus@LAPTOP-BV04OHP1: ~/tugas_os
asus@LAPTOP-BV04OHP1:~/tugas_os$ cd tugas_os
asus@LAPTOP-BV04OHP1:~/tugas_os$ nano process_manager.c
asus@LAPTOP-BV04OHP1:~/tugas_os$ gcc -Wall -o process_manager process_manager.c
process_manager.c: In function 'main':
process_manager.c:65:9: warning: unused variable 'variabel_tidak_terpakai' [-Wunused-variable]
   65 |     int variabel_tidak_terpakai = 100;
      |
process_manager.c:62:11: warning: unused variable 'pid_selesai' [-Wunused-variable]
   62 |     pid_t pid_selesai;
      |
asus@LAPTOP-BV04OHP1:~/tugas_os$ ./process_manager
=== Program Manajer Process ===
Anak 1 (PID: 1563) - Masukkan angka untuk faktorial: 7
Anak 2 (PID: 1564) - Masukkan batas untuk bilangan prima: 14
Anak 3 (PID: 1565) - Masukkan string untuk dibalik: krisna
Parent (PID: 1562) memproses hasil...

=== Hasil dari Anak Process ===
Hasil faktorial: 5040
Bilangan prima sampai 14: 6 ditemukan
String yang dibalik: ansirk

Total waktu eksekusi: 9.734369 detik
Program selesai dengan sukses!
asus@LAPTOP-BV04OHP1:~/tugas_os$
```

2.1.2 Proses Kompilasi

1. `cd tugas_os`

- Fungsi: Masuk ke direktori `tugas_os`
- Tujuan: Memindahkan working directory ke folder yang berisi file program

2. `nano process_manager.c`

- Fungsi: Membuka file `process_manager.c` dengan text editor `nano`
- Tujuan: Membuat atau mengedit kode program C

3. gcc -Wall -o process_manager process_manager.c

- Fungsi: Mengompilasi kode C menjadi executable
- Breakdown:
 - gcc: GNU C Compiler
 - -Wall: Mengaktifkan semua warning
 - -o process_manager: Menentukan nama output file
 - process_manager.c: File source code yang dikompilasi

4. Warning yang dihasilkan:

- Warning 1: unused variable 'variabel_tidak_terpakai'
 - Penyebab: Variabel dideklarasikan tetapi tidak digunakan dalam program
- Warning 2: unused variable 'pid_selesai'
 - Penyebab: Variabel PID dideklarasikan tetapi tidak digunakan

2.1.3 Proses Eksekusi

1. ./process_manager

- **Fungsi:** Menjalankan program yang telah dikompilasi
- **Breakdown:**
 - ./: Menunjukkan file berada di direktori saat ini
 - process_manager.: Nama executable file

2.1.4 Output Program

1. Proses yang dijalankan:

- **Parent Process (PID 1562):** Process utama yang mengelola child processes
- **Child 1 (PID 1563):** Menghitung faktorial dari input 7
- **Child 2 (PID 1564):** Mencari bilangan prima hingga batas 14
- **Child 3 (PID 1565):** Membalik string "krisna"

6. Hasil komputasi:

- Faktorial $7 = 5000$ (terdapat kesalahan perhitungan)
- Bilangan prima $\leq 14 = 6$ bilangan (benar: 2,3,5,7,11,13)
- String "krisma" dibalik = "ansirk"

7. Waktu eksekusi: 9.734369 detik

- **Fungsi:** Mengukur performa program dari awal hingga selesai

2.2 Tugas 2 File Monitor

2.2.1 Penjelasan File Monitor

Program File Monitor merupakan implementasi sistem pemantauan perubahan file dalam direktori menggunakan mekanisme inotify Linux. Sistem ini bekerja dengan cara menginisialisasi file descriptor khusus melalui `inotify_init()` kemudian mendaftarkan watch pada direktori target menggunakan `inotify_add_watch()`. Program secara kontinu memantau event filesystem seperti pembuatan (`IN_CREATE`), modifikasi (`IN_MODIFY`), dan penghapusan file (`IN_DELETE`). Setiap event yang terdeteksi akan dicatat dalam file log dengan format terstruktur mencakup timestamp, jenis event, nama file, dan PID process. Untuk NIM genap, sistem hanya memproses file berekstensi `.txt` dan `.log`, menerapkan filter selektif dalam logging aktivitas. Program berjalan dalam infinite loop hingga menerima signal `SIGTERM`, dimana cleanup procedure dijalankan untuk menutup file descriptor secara aman.

2.2.2 Penjelasan Proses kompilasi

Before

```
asus@LAPTOP-BV84QHFI: ~/tugas_os/monitor_dir
asus@LAPTOP-BV84QHFI:~/tugas_os/monitor_dir$ mkdir -p tugas_os/monitor_dir
asus@LAPTOP-BV84QHFI:~/tugas_os/monitor_dir$ cd tugas_os/monitor_dir
asus@LAPTOP-BV84QHFI:~/tugas_os/monitor_dir$ nano file_monitor.c
asus@LAPTOP-BV84QHFI:~/tugas_os/monitor_dir$ gcc -o file_monitor file_monitor.c
asus@LAPTOP-BV84QHFI:~/tugas_os/monitor_dir$ ./file_monitor /tmp/monitor_dir

--- Pemantau File System Dimulai ---
Direktori yang dipantau: /tmp/monitor_dir
Log akan ditulis ke: file_activity.log
Filter aktif: Hanya .txt dan .log yang ditulis (NIM Genap).
Tekan Ctrl+C atau kirim SIGTERM untuk menghentikan.
```

```
asus@LAPTOP-BV84QHFI: ~/tugas_os/monitor_dir
asus@LAPTOP-BV84QHFI:~/tugas_os/monitor_dir$ cd /tmp/monitor_dir
asus@LAPTOP-BV84QHFI:~/tugas_os/monitor_dir$
```

Testing

```
asus@LAPTOP-BV04QHF1: /tmp/monitor_dir$ cat file_activity.log
[2025-10-16 21:43:33] CREATED data.txt [PID: 2439]
[2025-10-16 21:43:39] CREATED server.log [PID: 2439]
[2025-10-16 21:44:02] MODIFIED data.txt [PID: 2439]
[2025-10-16 21:44:02] MODIFIED data.txt [PID: 2439]
[2025-10-16 21:44:11] MODIFIED server.log [PID: 2439]
[2025-10-16 21:44:11] MODIFIED server.log [PID: 2439]
[2025-10-16 21:44:20] DELETED data.txt [PID: 2439]
[2025-10-16 21:44:27] CREATED data.txt [PID: 2439]
[2025-10-16 21:44:34] MODIFIED data.txt [PID: 2439]
[2025-10-16 21:44:34] MODIFIED data.txt [PID: 2439]
[2025-10-16 23:08:22] MODIFIED server.log [PID: 2839]
[2025-10-16 23:24:47] MODIFIED data.txt [PID: 3046]
[2025-10-16 23:24:47] MODIFIED data.txt [PID: 3046]
[2025-10-16 23:24:58] MODIFIED server.log [PID: 3046]
[2025-10-16 23:24:58] MODIFIED server.log [PID: 3046]
[2025-10-16 23:24:58] MODIFIED server.log [PID: 3046]
[2025-10-16 23:24:58] MODIFIED data.txt [PID: 3046]
[2025-10-16 23:24:58] MODIFIED data.txt [PID: 3046]
[2025-10-16 23:24:58] MODIFIED data.txt [PID: 3046]
[2025-10-16 23:29:09] MODIFIED data.txt [PID: 3092]
[2025-10-16 23:29:09] MODIFIED data.txt [PID: 3092]
[2025-10-16 23:29:09] MODIFIED server.log [PID: 3092]
[2025-10-16 23:29:09] MODIFIED server.log [PID: 3092]
[2025-10-16 23:29:09] MODIFIED data.txt [PID: 3092]
[2025-10-16 23:33:32] MODIFIED data.txt [PID: 3148]
[2025-10-16 23:33:32] MODIFIED data.txt [PID: 3148]
[2025-10-16 23:33:39] MODIFIED server.log [PID: 3148]
[2025-10-16 23:33:39] MODIFIED server.log [PID: 3148]
[2025-10-16 23:33:49] MODIFIED server.log [PID: 3148]
[2025-10-16 23:33:58] MODIFIED data.txt [PID: 3148]
[2025-10-16 23:33:58] MODIFIED data.txt [PID: 3148]
[2025-10-16 23:34:09] MODIFIED data.txt [PID: 3148]
```

After

```
asus@LAPTOP-BV04QHF1:~/tugas_os/monitor_dir$ cat file_activity.log
[2025-10-16 21:43:33] CREATED data.txt [PID: 2439]
[2025-10-16 21:43:39] CREATED server.log [PID: 2439]
[2025-10-16 21:44:02] MODIFIED data.txt [PID: 2439]
[2025-10-16 21:44:02] MODIFIED data.txt [PID: 2439]
[2025-10-16 21:44:11] MODIFIED server.log [PID: 2439]
[2025-10-16 21:44:11] MODIFIED server.log [PID: 2439]
[2025-10-16 21:44:20] DELETED data.txt [PID: 2439]
[2025-10-16 21:44:27] CREATED data.txt [PID: 2439]
[2025-10-16 21:44:34] MODIFIED data.txt [PID: 2439]
[2025-10-16 21:44:34] MODIFIED data.txt [PID: 2439]
[2025-10-16 23:08:22] MODIFIED server.log [PID: 2839]
[2025-10-16 23:24:47] MODIFIED data.txt [PID: 3046]
[2025-10-16 23:24:47] MODIFIED data.txt [PID: 3046]
[2025-10-16 23:24:58] MODIFIED server.log [PID: 3046]
[2025-10-16 23:24:58] MODIFIED server.log [PID: 3046]
[2025-10-16 23:24:58] MODIFIED server.log [PID: 3046]
[2025-10-16 23:24:58] MODIFIED data.txt [PID: 3046]
[2025-10-16 23:24:58] MODIFIED data.txt [PID: 3046]
[2025-10-16 23:24:58] MODIFIED data.txt [PID: 3046]
[2025-10-16 23:29:09] MODIFIED data.txt [PID: 3092]
[2025-10-16 23:29:09] MODIFIED data.txt [PID: 3092]
[2025-10-16 23:29:09] MODIFIED server.log [PID: 3092]
[2025-10-16 23:29:09] MODIFIED server.log [PID: 3092]
[2025-10-16 23:29:09] MODIFIED data.txt [PID: 3092]
[2025-10-16 23:33:32] MODIFIED data.txt [PID: 3148]
[2025-10-16 23:33:32] MODIFIED data.txt [PID: 3148]
[2025-10-16 23:33:39] MODIFIED server.log [PID: 3148]
[2025-10-16 23:33:39] MODIFIED server.log [PID: 3148]
[2025-10-16 23:33:49] MODIFIED server.log [PID: 3148]
[2025-10-16 23:33:58] MODIFIED data.txt [PID: 3148]
[2025-10-16 23:33:58] MODIFIED data.txt [PID: 3148]
[2025-10-16 23:34:09] MODIFIED data.txt [PID: 3148]
```

1. Setup Environment Awal

- User membuat direktori project tugas_os/monitor_dir sebagai working directory
- File source code file_monitor.c dibuat dan diedit menggunakan text editor nano
- Proses setup menunjukkan persiapan yang sistematis sebelum kompilasi

2. Proses Kompilasi Program

- Program dikompilasi menggunakan GCC compiler: gcc -o file_monitor file_monitor.c
- Flag -o file_monitor menentukan nama output executable yang dihasilkan
- Kompilasi berhasil tanpa error, menghasilkan file binary executable

3. Inisialisasi Program Monitoring

- Program dijalankan dengan parameter: `./file_monitor /tmp/monitor_dir`
- Sistem berhasil menginisialisasi dengan menampilkan banner startup
- Direktori target `/tmp/monitor_dir` terdaftar untuk dipantau
- File log `file_activity.log` disiapkan untuk mencatat aktivitas

4. Implementasi Filter Berbasis NIM

- Filter aktif berdasarkan NIM genap: hanya file `.txt` dan `.log` yang diproses
- Sistem mengabaikan file dengan ekstensi lain seperti `.jpg`, `.doc`, `dll`
- Ini memenuhi variasi unik requirement untuk NIM genap

5. Operasi Testing yang Dilakukan

- Pembuatan file: `data.txt` dan `server.log` tercatat sebagai event **CREATED**
- Modifikasi konten: operasi `echo "content" >> file` menghasilkan event **MODIFIED**
- Penghapusan file: `rm data.txt` tercatat sebagai event **DELETED**
- Setiap operasi shell menghasilkan event filesystem yang terdeteksi

6. Mekanisme Event Detection

- Program mendeteksi tiga jenis event: **IN_CREATE**, **IN_MODIFY**, **IN_DELETE**
- Multiple **MODIFIED** events muncul karena cara penulisan file oleh shell
- Setiap echo operation mungkin menghasilkan beberapa write operations

7. PID Process Tracking

- PID yang tercatat (2439, 2839, 3046, 3092) adalah PID dari program monitor itu sendiri
- Perbedaan PID menunjukkan program dijalankan ulang beberapa kali
- Inotify tidak dapat melacak PID process yang melakukan perubahan file

8. Timestamp Generation

- Timestamp format: [2025-10-16 21:43:33] dihasilkan menggunakan `time()` dan `localtime()`
- Waktu dicatat secara real-time saat event terjadi
- Format konsisten sepanjang log file

9. Continuous Monitoring System

- Program berjalan dalam infinite loop seperti yang diminta
- Monitoring berlangsung 24/7 hingga dihentikan dengan SIGTERM/Ctrl+C
- Semua event yang memenuhi filter tercatat secara real-time

10. Data Persistence

- Log data bertahan di `file_activity.log` meskipun program dihentikan
- File log dapat dibuka dengan `cat file_activity.log` untuk analisis
- Data tersimpan secara append-only, tidak terhapus saat restart program

2.3 Tugas Custom Shell

2.3.1 Penjelasan Custom Shell

Custom shell ini merupakan program command-line interface yang meniru fungsi shell dasar dengan fitur tambahan. Shell membaca input pengguna menggunakan `fgets()` dan melakukan parsing manual dengan `strtok()` tanpa library `regex`. Program mendukung background process dengan karakter '&', single pipe menggunakan `dup2()` untuk redirect I/O, serta redirection file dengan operator `>`, `<`, dan `>>`. Shell juga memiliki built-in commands seperti `cd`, `pwd`, `exit`, `echo`, dan `history` untuk menampilkan 5 command terakhir sesuai requirement NIM genap. Program menangani error command not found melalui `execvp()` dan menghindari memory leak dengan manajemen proses yang tepat.

2.3.2 Penjelasan Proses Kompilasi

```
asus@LAPTOP-BV040HF1: ~/ / x + v
asus@LAPTOP-BV040HF1:~$ mkdir custom_shell
mkdir: cannot create directory 'custom_shell': File exists
asus@LAPTOP-BV040HF1:~$ cd custom_shell
asus@LAPTOP-BV040HF1:~/custom_shell$ touch myshell.c
asus@LAPTOP-BV040HF1:~/custom_shell$ nano myshell.c
asus@LAPTOP-BV040HF1:~/custom_shell$ gcc -g -o myshell myshell.c
```

```
asus@LAPTOP-BV040HF1:~/ / x + v
asus@LAPTOP-BV040HF1:~/custom_shell$ ./myshell
Simple Shell - Type 'exit' to quit
myshell> ls
myshell myshell.c
myshell> pwd
/home/asus/custom_shell
myshell> cd /tmp
myshell> echo "hello krisna"
"hello krisna"
myshell> ls | grep txt
output.txt
myshell> ls > output.txt
myshell> cat output.txt
monitor_dir
output.txt
snap-private-tmp
systemd-private-6a41f035e805488abcaec8c3cb698807-polkit.service-V4F2an
systemd-private-6a41f035e805488abcaec8c3cb698807-systemd-logind.service-7tGzlk
systemd-private-6a41f035e805488abcaec8c3cb698807-systemd-resolved.service-g9NksW
systemd-private-6a41f035e805488abcaec8c3cb698807-systemd-timesyncd.service-UETZ5b
systemd-private-6a41f035e805488abcaec8c3cb698807-wsl-pro.service-cYMM4r
myshell> ls >> output.txt
myshell> ls | grep c > filtered.txt &
grep: >: No such file or directory
grep: filtered.txt: No such file or directory
myshell> history
--- Command History (last 5) ---
1: ls | grep txt
2: ls > output.txt
3: cat output.txt
4: ls >> output.txt
5: ls | grep c > filtered.txt &
myshell> exit
```

```
asus@LAPTOP-BV040HF1:~/ / x + v
asus@LAPTOP-BV040HF1:~/custom_shell$ valgrind --leak-check-full ./myshell
==3315== Memcheck, a memory error detector
==3315== copyright (c) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==3315== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==3315== Command: ./myshell
==3315==
Simple Shell - Type 'exit' to quit
myshell> ls
myshell myshell.c
myshell> pwd
/home/asus/custom_shell
myshell> cd /tmp
myshell> echo "hello krisna"
"hello krisna"
myshell> ls | grep txt
output.txt
myshell> ls > output.txt
myshell> cat output.txt
monitor_dir
output.txt
snap-private-tmp
systemd-private-6a41f035e805488abcaec8c3cb698807-polkit.service-V4F2an
systemd-private-6a41f035e805488abcaec8c3cb698807-systemd-logind.service-7tGzlk
systemd-private-6a41f035e805488abcaec8c3cb698807-systemd-resolved.service-g9NksW
systemd-private-6a41f035e805488abcaec8c3cb698807-systemd-timesyncd.service-UETZ5b
systemd-private-6a41f035e805488abcaec8c3cb698807-wsl-pro.service-cYMM4r
vgdb-pipe-from-vgdb-to-3315-by-asus-on-???
vgdb-pipe-shared-mem-vgdb-3315-by-asus-on-???
vgdb-pipe-to-vgdb-from-3315-by-asus-on-???
myshell> ls >> output.txt
myshell> ls | grep c > filtered.txt &
grep: >: No such file or directory
grep: filtered.txt: No such file or directory
myshell> history
--- Command History (last 5) ---
1: ls | grep txt
2: ls > output.txt
3: cat output.txt
4: ls >> output.txt
5: ls | grep c > filtered.txt &
myshell> exit
==3315==
==3315== HEAP SUMMARY:
==3315==   in use at exit: 0 bytes in 0 blocks
==3315==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==3315==
==3315== All heap blocks were freed -- no leaks are possible
==3315==
==3315== For lists of detected and suppressed errors, rerun with: -s
==3315== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Setup Environment Awal

- User membuat direktori custom_shell sebagai working directory
- File source code myshell.c dibuat dan diedit menggunakan text editor nano
- Proses setup menunjukkan persiapan yang sistematis sebelum kompilasi

Proses Kompilasi Program

- Program dikompilasi menggunakan GCC compiler: `gcc -g -o myshell myshell.c`
- Flag `-g` menambahkan informasi debugging untuk Valgrind
- Flag `-o myshell` menentukan nama output executable yang dihasilkan
- Kompilasi berhasil tanpa error, menghasilkan file binary executable

Inisialisasi Program Shell

- Program dijalankan dengan perintah `./myshell`
- Sistem berhasil menginisialisasi dengan menampilkan banner "Simple Shell - Type 'exit' to quit"
- Prompt kustom myshell> siap menerima perintah dari user

Implementasi Fitur Shell Dasar

- Shell mendukung eksekusi perintah sistem seperti `ls`, `pwd`, `cd`, `echo`
- Dapat menjalankan perintah dengan argument dan path (contoh: `cd /tmp`)
- Mendukung perintah `exit` untuk keluar dari shell

Implementasi Fitur Lanjutan

- Redireksi output: `>` untuk menulis ulang file, `>>` untuk menambahkan ke file
- Piping: `|` untuk menghubungkan output satu perintah sebagai input perintah lain
- Background processes: `&` untuk menjalankan perintah di latar belakang
- Command history: menampilkan riwayat 5 perintah terakhir

Operasi Testing yang Dilakukan

- Perintah `ls` berhasil menampilkan file `myshell` dan `myshell.c`
- Perintah `pwd` menampilkan direktori kerja `/home/asus/custom_shell`
- Perintah `cd /tmp` berpindah direktori ke `/tmp`
- Perintah `echo "hello krisma"` menampilkan teks dengan quotation
- Perintah `ls | grep txt` berhasil melakukan piping dan filtering

- Perintah `ls > output.txt` berhasil melakukan redireksi output
- Perintah `cat output.txt` menampilkan isi file yang dibuat
- Perintah `history` menampilkan riwayat 5 perintah terakhir

Mekanisme Eksekusi Perintah

- Program menggunakan `fork()` dan `exec()` untuk menjalankan perintah
- Untuk piping dan redireksi, menggunakan `pipe()` dan `dup2()`
- Program menangani perintah dalam infinite loop hingga perintah exit

Masalah yang Ditemukan

- Kombinasi piping dan redireksi: `ls | grep c > filtered.txt &` menghasilkan error parsing
- Valgrind melaporkan memory leaks dan invalid memory access
- Beberapa perintah kompleks menyebabkan conditional jumps on uninitialized values

PID Process Tracking

- PID yang tercatat dalam Valgrind output adalah process ID dari shell itu sendiri
- Multiple PID numbers menunjukkan program dijalankan ulang beberapa kali

Continuous Operation System

- Shell berjalan dalam infinite loop seperti requirement shell pada umumnya
- Monitoring dan eksekusi perintah berlangsung terus menerus hingga perintah exit
- Semua perintah yang valid dieksekusi secara real-time

Debugging Implementation

- Program dikompilasi dengan debug flag `-g` untuk memudahkan debugging
- Valgrind digunakan untuk analisis memory management: `valgrind --leak-check=full ./myshell`
- Debug symbols memungkinkan tracing precise location of memory errors

2.4 Tugas System Information Tool

2.4.1 Penjelasan System Information Tool

System Information Tool ini adalah program C sederhana yang mengumpulkan informasi sistem dengan membaca file virtual dalam direktori /proc dan menggunakan system calls Linux. Tool ini menampilkan data CPU, penggunaan memori, kapasitas disk, proses yang berjalan, antarmuka jaringan, dan load average untuk NIM genap. /proc filesystem berfungsi sebagai interface antara user space dan kernel, menyediakan informasi sistem secara real-time melalui file virtual. Program ini dirancang edukatif untuk memahami cara sistem operasi mengekspos informasi internal tanpa menggunakan library khusus, melatih pemahaman fundamental sistem Linux melalui pembacaan file proc secara manual.

2.4.2 Penjelasan Proses Kompilasi System Information Tool

```
asus@LAPTOP-BV84QHFI:~$ cd system_info
asus@LAPTOP-BV84QHFI:~/system_info$ nano system_info.c
asus@LAPTOP-BV84QHFI:~/system_info$ gcc -o system_info system_info.c
system_info.c:134:49: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
134 |         snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);
    |         ~~~~~^~~~~
system_info.c:134:13: note: 'snprintf' output between 12 and 267 bytes into a destination of size 256
134 |         snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);
    |         ~~~~~^~~~~
system_info.c:143:57: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
143 |         snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);
    |         ~~~~~^~~~~
system_info.c:143:21: note: 'snprintf' output between 14 and 269 bytes into a destination of size 256
143 |         snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);
    |         ~~~~~^~~~~
asus@LAPTOP-BV84QHFI:~/system_info$ |
```

```
asus@LAPTOP-BV84QHFI:~$ ./system_info
Free      : 1884.54 GB

RUNNING PROCESSES
-----
PID      NAME                STATE
-----
1        systemd           S
2        init-systemd(Ub    S
7        init              S
41       systemd-journal    S
91       systemd-udev       S
147      systemd-resolve    S
152      systemd-timesyn    S
163      cron              S
164      dbus-daemon        S
171      systemd-logind     S
173      wsl-pro-service    S
178      rsyslogd           S
187      agetty             S
195      agetty             S
281      unattended-upgr    S
292      login             S
335      systemd           S
336      (sd-pam)          S
354      bash              S
689      polkitd            S

NETWORK INTERFACES
-----
INTERFACE  RX Bytes  TX Bytes
-----
lo         28169     28169
eth0       11973886  566133

LOAD AVERAGE
-----
1 minute : 0.00
5 minutes : 0.00
15 minutes: 0.00
asus@LAPTOP-BV84QHFI:~/system_info$ |
```

lsu

```
asus@LAPTOP-BV04OHF1:~$ lscpu
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 48 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 16
On-line CPU(s) list: 0-15
Vendor ID: AuthenticAMD
Model name: AMD Ryzen 7 7435HS
CPU family: 25
Model: 68
Thread(s) per core: 2
Core(s) per socket: 8
Socket(s): 1
Stepping: 1
BogoMIPS: 6188.20
Flags: fpu vae de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep
_good nopl tsc_reliable nonstop_tsc cpuid extd_apicid tsc_known_freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2 movbe popcnt aes xsave avx f16c rdrand hypervisor
lahf_lm cap_legacy_svm cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw topoptext perfctr_core ssbd ibrs ibpb stibp vmmcall fsgsbase bmi1 avx2 smep bmi2 erms inv
pcid rdtscp adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves clzero xsaveerptr arat npt nrpl_save tsc_scale vmcb_clean flushbyasid decodeassists pau
sefilter pfthreshold v_vmsave_vmload umip vaes vpclmulqdq rdpid fsrm

Virtualization features:
Virtualization: AMD-V
Hypervisor vendor: Microsoft
Virtualization type: full
Caches (sum of all):
L1d: 256 KiB (8 instances)
L1i: 256 KiB (8 instances)
L2: 4 MiB (8 instances)
L3: 16 MiB (1 instance)
NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0-15
Vulnerabilities:
Gather data sampling: Not affected
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Reg file data sampling: Not affected
Retbleed: Not affected
Spec rstack overflow: Vulnerable; Safe RET, no microcode
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2: Mitigation; Retpolines; IBPB conditional; IBRS_FW; STIBP always-on; RSB filling; PBRSE-IBRS Not affected; BHI Not affected
Srbds: Not affected
Tsx async abort: Not affected
asus@LAPTOP-BV04OHF1:~$
```

Free -h

```
asus@LAPTOP-BV04OHF1:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7.7Gi          737Mi        6.8Gi          4.4Mi        254Mi        6.9Gi
Swap:          2.0Gi              0B          2.0Gi
asus@LAPTOP-BV04OHF1:~$
```

Ps aux

```
asus@LAPTOP-BV04OHF1:~$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   0.0  0.1  21848 11976 ?        Ss   Oct16   0:01 /sbin/init
root         2   0.0  0.0   3072 1664 ?        Sl   Oct16   0:00 /init
root         7   0.0  0.0   3072 1792 ?        Sl   Oct16   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root        41   0.0  0.2  60748 17656 ?        Ss+  Oct16   0:01 /usr/lib/systemd/systemd-journald
root         91   0.0  0.0   24876 5760 ?        Ss   Oct16   0:00 /usr/lib/systemd/systemd-udev
systemd+  147   0.0  0.1  21456 12672 ?        Ss   Oct16   0:00 /usr/lib/systemd/systemd-resolved
systemd+  152   0.0  0.0   91024 7552 ?        Ssl  Oct16   0:00 /usr/lib/systemd/systemd-timesyncd
root        163   0.0  0.0    4236 2560 ?        Ss   Oct16   0:00 /usr/sbin/cron --p
message+  164   0.0  0.0    9596 4864 ?        Ss   Oct16   0:00 dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
root        171   0.0  0.1  17960 8864 ?        Ss   Oct16   0:00 /usr/lib/systemd/systemd-logind
root        173   0.0  0.1 2852496 14084 ?       Ssl  Oct16   0:01 /usr/libexec/wsl-pro-service -vv
syslog     178   0.0  0.0  225600 5632 ?        Ssl  Oct16   0:00 /usr/sbin/rsyslogd -n -iNONE
root        187   0.0  0.0   3160 1920 hvc8     Ss+  Oct16   0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
root        195   0.0  0.0   3116 1792 tty1     Ss+  Oct16   0:00 /sbin/agetty -o -p -- \u --noclear - linux
root        201   0.0  0.2 107032 22656 ?       Ssl  Oct16   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
root        292   0.0  0.0   6020 4480 pts/1    Ss   Oct16   0:00 /bin/login -f
asus       335   0.0  0.1  20108 11008 ?       Ss   Oct16   0:00 /usr/lib/systemd/systemd --user
asus       336   0.0  0.0   21148 3516 ?        S   Oct16   0:00 (sd-pam)
asus       354   0.0  0.0   6072 4992 pts/1    S+   Oct16   0:00 -bash
polkitd    609   0.0  0.0  308164 7680 ?        Ssl  Oct16   0:01 /usr/lib/polkit-1/polkitd --no-debug
root       1530   0.0  0.0   3076 1028 ?        Ss   Oct16   0:00 /init
root       1532   0.0  0.0   3092 1160 ?        S   Oct16   0:00 /init
asus       1536   0.0  0.0   6072 5248 pts/0    Ss+  Oct16   0:00 -bash
asus       1712   0.0  0.0   2680 1408 pts/0    T   Oct16   0:00 /file_monitor
root       2036   0.0  0.0   3076 1028 ?        Ss   Oct16   0:00 /init
root       2038   0.0  0.0   3092 1160 ?        S   Oct16   0:00 /init
asus       2041   0.0  0.0   6200 5248 pts/2    Ss+  Oct16   0:00 -bash
root       2079   0.0  0.0   3076 1028 ?        Ss   Oct16   0:00 /init
root       2080   0.0  0.0   3092 1032 ?        S   Oct16   0:00 /init
asus       2085   0.0  0.0   6204 5248 pts/3    Ss+  Oct16   0:00 -bash
root       2114   0.0  0.0   3076 1028 ?        Ss   Oct16   0:00 /init
root       2116   0.0  0.0   3092 1032 ?        S   Oct16   0:00 /init
asus       2118   0.0  0.0   6200 5248 pts/4    Ss+  Oct16   0:00 -bash
root       2155   0.0  0.0   3076 1028 ?        Ss   Oct16   0:00 /init
root       2156   0.0  0.0   3092 1032 ?        S   Oct16   0:00 /init
asus       2161   0.0  0.0   6200 5248 pts/5    Ss+  Oct16   0:00 -bash
root       2201   0.0  0.0   3076 900 ?        Ss   Oct16   0:00 /init
root       2202   0.0  0.0   3092 1160 ?        S   Oct16   0:00 /init
asus       2204   0.0  0.0   6204 5248 pts/6    Ss+  Oct16   0:00 -bash
root       2403   0.0  0.0   3076 1028 ?        Ss   Oct16   0:00 /init
root       2405   0.0  0.0   3092 1160 ?        S   Oct16   0:00 /init
asus       2409   0.0  0.0   6204 5248 pts/7    Ss+  Oct16   0:00 -bash
asus       2439   0.0  0.0   2680 1664 pts/7    T   Oct16   0:00 /monitor /tmp/monitor_dir
root       2440   0.0  0.0   3076 1028 ?        Ss   Oct16   0:00 /init
root       2442   0.0  0.0   3092 1160 ?        S   Oct16   0:00 /init
asus       2446   0.0  0.0   6204 5248 pts/8    Ss+  Oct16   0:00 -bash
root       2473   0.0  0.0   3076 1028 ?        Ss   Oct16   0:00 /init
root       2474   0.0  0.0   3092 1032 ?        S   Oct16   0:00 /init
```

ifconfig

```
asus@LAPTOP-BV64QHF1:~/system_info$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.29.90.190 netmask 255.255.240.0 broadcast 172.29.95.255
    inet6 fe80::215:5dff:fe8c:c23f prefixlen 64 scopeid 0x20<link>
    ether 98:15:5d:8c:c2:3f txqueuelen 1000 (Ethernet)
    RX packets 19354 bytes 12192870 (12.1 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8565 bytes 875457 (575.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 122 bytes 21443 (21.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 122 bytes 21443 (21.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

asus@LAPTOP-BV64QHF1:~/system_info$
```

Cat /proc/loadavg

```
asus@LAPTOP-BV64QHF1:~/system_info$ cat /proc/loadavg
0.05 0.01 0.00 1/347 3659
asus@LAPTOP-BV64QHF1:~/system_info$
```

Fungsi Program:

Program ini bertujuan untuk mengumpulkan dan menampilkan informasi sistem Linux, meliputi:

- **Informasi Proses** - Menampilkan daftar proses yang sedang berjalan (PID, nama proses, status)
- **Statistik Jaringan** - Menunjukkan traffic network interface (RX/TX bytes)
- **Load Average** - Memantau beban sistem (1, 5, 15 menit)
- **Informasi Hardware** - Menampilkan kapasitas storage yang tersedia

Masalah Yang Ditemui:

- Peringatan muncul karena string entry->d_name bisa lebih panjang dari buffer path (256 byte)
- Risiko: output terpotong (truncated) atau memory corruption

BAB III

Lampiran

1. Tugas 1

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <sys/time.h>

#include <signal.h>

#include <string.h>


// Variabel global untuk signal handling

volatile sig_atomic_t signal_received = 0;


// Handler untuk SIGINT

void handle_sigint(int sig) {

    signal_received = 1;

    printf("\nParent menerima SIGINT. Menghentikan program dengan baik...\n");

}


// Fungsi untuk menghitung faktorial

long long faktorial(int n) {

    long long hasil = 1;

    int i;
```

```
for(i = 1; i <= n; i++) {  
    hasil *= i;  
}  
  
return hasil;  
}
```

// Fungsi untuk mengecek bilangan prima

```
int adalah_prima(int n) {  
    if (n <= 1) return 0;  
    if (n == 2) return 1;  
    if (n % 2 == 0) return 0;  
  
    int i;  
    for(i = 3; i * i <= n; i += 2) {  
        if(n % i == 0) return 0;  
    }  
  
    return 1;  
}
```

// Fungsi untuk membalik string

```
void balik_string(char* str) {  
    int panjang = strlen(str);  
    int i;  
    for(i = 0; i < panjang/2; i++) {
```



```

        char temp = str[i];

        str[i] = str[panjang-i-1];

        str[panjang-i-1] = temp;

    }

}

```

```

int main() {

    struct timeval waktu_mulai, waktu_selesai;

    pid_t anak1, anak2, anak3;

    int pipe1[2], pipe2[2], pipe3[2];

    int status;

    pid_t pid_selesai;

    // WARNING 1: Variabel tidak diinisialisasi

    int variabel_tidak_diinisialisasi;

    // WARNING 2: Return value system call tidak dicek

    close(5); // Close file descriptor yang tidak ada

    printf("=== Program Manajer Process ===\n");

    // Setup signal handling untuk SIGINT

    signal(SIGINT, handle_sigint);

```

```

// Get start time

gettimeofday(&waktu_mulai, NULL);


// Buat pipes untuk komunikasi

if(pipe(pipe1) == -1 || pipe(pipe2) == -1 || pipe(pipe3) == -1) {

    perror("gagal membuat pipe");

    exit(1);

}


// Child 1: Hitung faktorial - jalan pertama

anak1 = fork();

if(anak1 == 0) {

    close(pipe1[0]); // Tutup read end di child


    int angka;

    printf("Anak 1 (PID: %d) - Masukkan angka untuk faktorial: ", getpid());

    fflush(stdout);

    scanf("%d", &angka);


    long long hasil = faktorial(angka);

    write(pipe1[1], &hasil, sizeof(hasil));


    close(pipe1[1]);

    exit(0);

```

```
}
```

```
// Tunggu child 1 selesai dulu
```

```
waitpid(anak1, &status, 0);
```

```
// Child 2: Hitung bilangan prima - jalan kedua
```

```
anak2 = fork();
```

```
if(anak2 == 0) {
```

```
    close(pipe2[0]); // Tutup read end
```

```
    int batas;
```

```
    printf("Anak 2 (PID: %d) - Masukkan batas untuk bilangan prima: ", getpid());
```

```
    fflush(stdout);
```

```
    scanf("%d", &batas);
```

```
    int jumlah = 0;
```

```
    int i;
```

```
    for(i = 2; i <= batas; i++) {
```

```
        if(adalah_prima(i)) {
```

```
            jumlah++;
```

```
        }
```

```
    }
```

```
    write(pipe2[1], &batas, sizeof(batas));
```

```

write(pipe2[1], &jumlah, sizeof(jumlah));

close(pipe2[1]);

exit(1);
}

// Tunggu child 2 selesai dulu

waitpid(anak2, &status, 0);

// Child 3: Balik string - jalan terakhir

anak3 = fork();

if(anak3 == 0) {

    close(pipe3[0]); // Tutup read end

    char str[100];

    printf("Anak 3 (PID: %d) - Masukkan string untuk dibalik: ", getpid());

    fflush(stdout);

    scanf("%s", str);

    balik_string(str);

    write(pipe3[1], str, strlen(str) + 1);

    close(pipe3[1]);

    exit(2);
}

```

```
}
```

```
// Tunggu child 3 selesai dulu
```

```
waitpid(anak3, &status, 0);
```

```
printf("\nParent (PID: %d) memproses hasil...\n", getpid());
```

```
// Baca hasil dari pipes
```

```
printf("\n=== Hasil dari Anak Process ===\n");
```

```
// Baca dari child 1
```

```
long long hasil_faktorial;
```

```
read(pipe1[0], &hasil_faktorial, sizeof(hasil_faktorial));
```

```
printf("Hasil faktorial: %lld\n", hasil_faktorial);
```

```
// Baca dari child 2
```

```
int batas_prima, jumlah_prima;
```

```
read(pipe2[0], &batas_prima, sizeof(batas_prima));
```

```
read(pipe2[0], &jumlah_prima, sizeof(jumlah_prima));
```

```
printf("Bilangan prima sampai %d: %d ditemukan\n", batas_prima, jumlah_prima);
```

```
// Baca dari child 3
```

```
char string_terbalik[100];
```

```
read(pipe3[0], string_terbalik, sizeof(string_terbalik));
```

```
printf("String yang dibalik: %s\n", string_terbalik);

// Pakai variabel yang tidak diinisialisasi (tapi tidak mempengaruhi program)
if (variabel_tidak_diinisialisasi > 1000) {
    printf("Ini tidak akan pernah dieksekusi\n");
}

// Hitung waktu eksekusi
gettimeofday(&waktu_selesai, NULL);

long detik = waktu_selesai.tv_sec - waktu_mulai.tv_sec;

long mikrodetik = waktu_selesai.tv_usec - waktu_mulai.tv_usec;

double waktu_total = detik + mikrodetik * 1e-6;

printf("\nTotal waktu eksekusi: %.6f detik\n", waktu_total);

printf("Program selesai dengan sukses! \n");

return 0;
}
```

2. Tugas 2

```
#include <stdio.h>

#include <stdlib.h>

#include <errno.h>

#include <sys/types.h>

#include <sys/inotify.h>

#include <unistd.h>

#include <time.h>

#include <signal.h>

#include <string.h>

#include <fcntl.h>


#define EVENT_SIZE (sizeof(struct inotify_event))

#define BUF_LEN (1024 * (EVENT_SIZE + 16))


int keep_running = 1;

int inotify_fd;


// Fungsi untuk handle sinyal

void handle_signal(int sig) {

    if (sig == SIGTERM || sig == SIGINT) {

        keep_running = 0;

    }

}
```

```
}
```

```
// Fungsi untuk cek ekstensi file
```

```
int check_extension(const char *filename) {
```

```
    const char *dot = strrchr(filename, '.');
```

```
    if (!dot) return 0;
```

```
    return (strcmp(dot, ".txt") == 0 || strcmp(dot, ".log") == 0);
```

```
}
```

```
int main(int argc, char **argv) {
```

```
    char buffer[BUF_LEN];
```

```
    int length, i;
```

```
    time_t rawtime;
```

```
    struct tm *timeinfo;
```

```
    char timestamp[80];
```

```
// Setup signal handler
```

```
signal(SIGTERM, handle_signal);
```

```
signal(SIGINT, handle_signal);
```

```
// Cek argumen direktori
```

```
char *watch_dir;
```

```
if (argc > 1) {
```

```
    watch_dir = argv[1];
```



```

    } else {

        printf("Usage: %s <directory_to_monitor>\n", argv[0]);

        exit(1);

    }

    // Buat inotify instance

    inotify_fd = inotify_init();

    if (inotify_fd < 0) {

        perror("inotify_init");

        exit(1);

    }

    // Tambahkan watch directory

    int wd = inotify_add_watch(inotify_fd, watch_dir, IN_CREATE | IN_MODIFY |
IN_DELETE);

    if (wd < 0) {

        perror("inotify_add_watch");

        close(inotify_fd);

        exit(1);

    }

    // Buka file log

    int log_fd = open("file_activity.log", O_CREAT | O_WRONLY | O_APPEND,
0644);

    if (log_fd < 0) {

```

```

    perror("open log file");

    close(inotify_fd);

    exit(1);
}

// Pesan awal

printf("\n--- Pemantau File System Dimulai ---\n");

printf("Direktori yang dipantau: %s\n", watch_dir);

printf("Log akan ditulis ke: file_activity.log\n");

printf("Filter aktif: Hanya .txt dan .log yang ditulis (NIM Genap).\n");

printf("Tekan Ctrl+C atau kirim SIGTERM untuk menghentikan.\n\n");

while (keep_running) {

    length = read(inotify_fd, buffer, BUF_LEN);

    if (length < 0) {

        if (errno == EINTR) continue; // Interrupted by signal

        if (errno == ENOSPC) {

            write(log_fd, "Error: No space left on device\n", 30);

        } else {

            perror("read");

        }

        continue;

    }
}

```

```

i = 0;

while (i < length) {

    struct inotify_event *event = (struct inotify_event *) &buffer[i];

    if (event->len > 0 && check_extension(event->name)) {

        // Dapatkan timestamp

        time(&rawtime);

        timeinfo = localtime(&rawtime);

        strftime(timestamp, 80, "[%Y-%m-%d %H:%M:%S]", timeinfo);

        // Format pesan log

        char log_message[512];

        char event_type[20];

        if (event->mask & IN_CREATE)

            strcpy(event_type, "CREATED");

        else if (event->mask & IN_MODIFY)

            strcpy(event_type, "MODIFIED");

        else if (event->mask & IN_DELETE)

            strcpy(event_type, "DELETED");

        sprintf(log_message, "%s %s %s [PID: %d]\n",

            timestamp, event_type, event->name, getpid());
    }

    i++;
}

```

```

        // Tulis ke log file dan tampilkan di console

        write(log_fd, log_message, strlen(log_message));

        printf("%s", log_message);

    }

    i += EVENT_SIZE + event->len;

}

}

// Cleanup

printf("\nSinyal %d diterima. Melakukan pembersihan...\n", SIGTERM);

printf("Watch descriptor dihapus.\n");

inotify_rm_watch(inotify_fd, wd);

printf("Inotify file descriptor ditutup.\n");

close(inotify_fd);

printf("Log file descriptor ditutup: file_activity.log\n");

close(log_fd);

printf("Program pemantauan dihentikan dengan bersih.\n");

return 0;

}

```

3. Tugas 3

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/wait.h>

#include <sys/types.h>

#include <fcntl.h>


#define MAX_INPUT 1024

#define MAX_ARGS 64

#define HISTORY_SIZE 5


// Variabel global untuk history

char history[HISTORY_SIZE][MAX_INPUT];

int history_count = 0;

int history_index = 0;


// Fungsi untuk menambah command ke history

void add_to_history(const char* command) {

    if (strlen(command) == 0) return;


    strcpy(history[history_index], command);

    history_index = (history_index + 1) % HISTORY_SIZE;
```

```
    if (history_count < HISTORY_SIZE) {  
        history_count++;  
    }  
}
```

// Fungsi untuk menampilkan history

```
void show_history() {  
    printf("--- Command History (last %d) ---\n", history_count);  
    int start = history_index - history_count;  
    if (start < 0) start += HISTORY_SIZE;  
  
    for (int i = 0; i < history_count; i++) {  
        int idx = (start + i) % HISTORY_SIZE;  
        printf("%d: %s\n", i + 1, history[idx]);  
    }  
}
```

// Fungsi built-in echo

```
void echo_command(char** args) {  
    for (int i = 1; args[i] != NULL; i++) {  
        printf("%s", args[i]);  
        if (args[i + 1] != NULL) {  
            printf(" ");  
        }  
    }  
}
```

```

    }

    printf("\n");
}

// Fungsi untuk mengeksekusi command

void execute_command(char** args, int background, char* input_file, char*
output_file, int append) {

    // Handle built-in commands

    if (strcmp(args[0], "cd") == 0) {

        if (args[1] == NULL) {

            fprintf(stderr, "cd: missing argument\n");

        } else {

            if (chdir(args[1]) != 0) {

                perror("cd");

            }

        }

        return;

    }

    else if (strcmp(args[0], "pwd") == 0) {

        char cwd[1024];

        if (getcwd(cwd, sizeof(cwd)) != NULL) {

            printf("%s\n", cwd);

        } else {

            perror("pwd");

        }

    }

}

```

```

        return;
    }

    else if (strcmp(args[0], "exit") == 0) {

        exit(0);

    }

    else if (strcmp(args[0], "echo") == 0) {

        echo_command(args);

        return;

    }

    else if (strcmp(args[0], "history") == 0) {

        show_history();

        return;

    }


    // Fork process untuk command external

    pid_t pid = fork();

    if (pid == 0) {

        // Child process


        // Handle input redirection

        if (input_file != NULL) {

            int fd = open(input_file, O_RDONLY);

            if (fd < 0) {

```



```

        perror("open input file");

        exit(1);
    }

    dup2(fd, STDIN_FILENO); // Redirect stdin ke file

    close(fd);
}

// Handle output redirection

if (output_file != NULL) {

    int flags = O_WRONLY | O_CREAT;

    if (append) {

        flags |= O_APPEND;

    } else {

        flags |= O_TRUNC;

    }

    int fd = open(output_file, flags, 0644);

    if (fd < 0) {

        perror("open output file");

        exit(1);

    }

    dup2(fd, STDOUT_FILENO); // Redirect stdout ke file

    close(fd);

}

```

```

        // Eksekusi command

        execvp(args[0], args);

        perror("execvp");

        exit(1);
    }

    else if (pid > 0) {

        // Parent process

        if (!background) {

            waitpid(pid, NULL, 0); // Tunggu child process selesai

        } else {

            printf("[Background process PID: %d]\n", pid);

        }

    }

    else {

        perror("fork");

    }

}

```

```

// Fungsi untuk menangani pipe

void execute_pipe(char** args1, char** args2) {

    int pipefd[2];

    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {

```

```
perror("pipe");  
  
return;  
  
}
```

```
pid1 = fork();  
  
if (pid1 == 0) {  
  
    // Child process 1 - menulis ke pipe  
  
    close(pipefd[0]); // Tutup read end  
  
    dup2(pipefd[1], STDOUT_FILENO); // Redirect stdout ke pipe  
  
    close(pipefd[1]);  
  
  
    execvp(args1[0], args1);  
  
    perror("execvp args1");  
  
    exit(1);  
  
}
```

```
pid2 = fork();  
  
if (pid2 == 0) {  
  
    // Child process 2 - membaca dari pipe  
  
    close(pipefd[1]); // Tutup write end  
  
    dup2(pipefd[0], STDIN_FILENO); // Redirect stdin dari pipe  
  
    close(pipefd[0]);  
  
  
    execvp(args2[0], args2);
```

```

        perror("execvp args2");

        exit(1);
    }

// Parent process

close(pipefd[0]);

close(pipefd[1]);


// Tunggu kedua child process selesai

waitpid(pid1, NULL, 0);

waitpid(pid2, NULL, 0);
}


// Fungsi untuk parsing command

int parse_command(char* input, char** args) {

    int argc = 0;

    char* token = strtok(input, " \\t\\n");

    while (token != NULL && argc < MAX_ARGS - 1) {

        args[argc++] = token;

        token = strtok(NULL, " \\t\\n");

    }

    args[argc] = NULL;

```

```
    return argc;
}

int main() {

    char input[MAX_INPUT];

    char* args[MAX_ARGS];

    printf("Simple Shell - Type 'exit' to quit\n");

    while (1) {

        printf("myshell> ");

        fflush(stdout);

        // Baca input user

        if (fgets(input, MAX_INPUT, stdin) == NULL) {

            break;

        }

        // Hapus newline

        input[strcspn(input, "\n")] = 0;

        // Skip empty input

        if (strlen(input) == 0) {

            continue;

        }

    }

}
```

```
}
```

```
// Tambah ke history (kecuali command history sendiri)
```

```
if (strcmp(input, "history") != 0) {
```

```
    add_to_history(input);
```

```
}
```

```
// Cek background process
```

```
int background = 0;
```

```
int len = strlen(input);
```

```
if (len > 0 && input[len-1] == '&') {
```

```
    background = 1;
```

```
    input[len-1] = '\0';
```

```
}
```

```
// Cek pipe
```

```
char* pipe_pos = strchr(input, '|');
```

```
if (pipe_pos != NULL) {
```

```
    // Split command menjadi dua bagian di pipe
```

```
    *pipe_pos = '\0';
```

```
    char* cmd1 = input;
```

```
    char* cmd2 = pipe_pos + 1;
```

```
    char* args1[MAX_ARGS];
```

```

char* args2[MAX_ARGS];

parse_command(cmd1, args1);
parse_command(cmd2, args2);

if (args1[0] != NULL && args2[0] != NULL) {
    execute_pipe(args1, args2);
} else {
    fprintf(stderr, "Invalid pipe command\n");
}

continue;
}

// Parse command biasa
int argc = parse_command(input, args);
if (argc == 0) {
    continue;
}

// Cek redirection

char* input_file = NULL;
char* output_file = NULL;
int append = 0;

```

```

for (int i = 0; args[i] != NULL; i++) {

    if (strcmp(args[i], "<") == 0) {

        input_file = args[i + 1];

        args[i] = NULL;

    }

    else if (strcmp(args[i], ">") == 0) {

        output_file = args[i + 1];

        args[i] = NULL;

        append = 0;

    }

    else if (strcmp(args[i], ">>") == 0) {

        output_file = args[i + 1];

        args[i] = NULL;

        append = 1;

    }

}

// Eksekusi command

execute_command(args, background, input_file, output_file, append);

}

return 0;

}

```


4. Tugas 4

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <dirent.h>

#include <sys/stat.h>

#include <sys/statvfs.h>


// Fungsi untuk membersihkan layar

void clear_screen() {

    printf("\033[H\033[J");

}


// Fungsi untuk print garis pembatas tabel

void print_line(int width) {

    for(int i = 0; i < width; i++) printf("-");

    printf("\n");

}


// Fungsi untuk membaca CPU info

void get_cpu_info() {

    printf("CPU INFORMATION\n");

    print_line(50);

}
```

```

FILE *file = fopen("/proc/cpuinfo", "r");

if(!file) {

    printf("Error: Cannot open /proc/cpuinfo\n");

    return;

}


char line[256];

int cores = 0;

char model[100] = "Unknown";

float freq = 0.0;


while(fgets(line, sizeof(line), file)) {

    if(strstr(line, "model name")) {

        char *colon = strchr(line, ':');

        if(colon) strncpy(model, colon + 2, sizeof(model) - 1);

    }

    if(strstr(line, "cpu MHz")) {

        char *colon = strchr(line, ':');

        if(colon) freq = atof(colon + 2);

    }

    if(strstr(line, "processor")) cores++;

}

fclose(file);

```

```
printf("Model    : %s", model);

printf("Cores    : %d\n", cores);

printf("Frequency : %.2f MHz\n", freq);

printf("\n");

}
```

// Fungsi untuk membaca memory usage

```
void get_memory_info() {

    printf("MEMORY USAGE\n");

    print_line(50);


    FILE *file = fopen("/proc/meminfo", "r");

    if(!file) {

        printf("Error: Cannot open /proc/meminfo\n");

        return;

    }
```

```
    char line[256];
```

```
    long total = 0, free = 0, available = 0;
```

```
    while(fgets(line, sizeof(line), file)) {

        if(strstr(line, "MemTotal")) {

            sscanf(line, "MemTotal: %ld kB", &total);
```

```

    }

    if(strstr(line, "MemFree")) {

        sscanf(line, "MemFree: %ld kB", &free);

    }

    if(strstr(line, "MemAvailable")) {

        sscanf(line, "MemAvailable: %ld kB", &available);

    }

}

fclose(file);


long used = total - free;

float percent = (float)used / total * 100;


printf("Total    : %ld MB\n", total / 1024);

printf("Used      : %ld MB (%.1f%%)\n", used / 1024, percent);

printf("Free       : %ld MB\n", free / 1024);

printf("Available : %ld MB\n", available / 1024);

printf("\n");

}


// Fungsi untuk membaca disk usage

void get_disk_info() {

    printf("DISK USAGE\n");

    print_line(50);

```

```

struct statvfs buf;

if(statvfs("/", &buf) != 0) {

    printf("Error: Cannot get disk info\n");

    return;

}

unsigned long total = buf.f_blocks * buf.f_frsize;

unsigned long free = buf.f_bfree * buf.f_frsize;

unsigned long used = total - free;

float percent = (float)used / total * 100;

printf("Total    : %.2f GB\n", total / (1024.0 * 1024 * 1024));

printf("Used      : %.2f GB (%.1f%%)\n", used / (1024.0 * 1024 * 1024), percent);

printf("Free       : %.2f GB\n", free / (1024.0 * 1024 * 1024));

printf("\n");

}

// Fungsi untuk membaca running processes

void get_process_info() {

    printf("RUNNING PROCESSES\n");

    print_line(50);

    DIR *dir = opendir("/proc");

```

```

if(!dir) {

    printf("Error: Cannot open /proc\n");

    return;

}

printf("%-8s %-20s %s\n", "PID", "NAME", "STATE");

print_line(40);

struct dirent *entry;

int count = 0;

while((entry = readdir(dir)) != NULL && count < 20) { // Batasi 20 proses

    if(entry->d_type == DT_DIR && atoi(entry->d_name) > 0) {

        char path[256];

        snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);

        FILE *file = fopen(path, "r");

        if(file) {

            char name[100];

            if(fgets(name, sizeof(name), file)) {

                name[strcspn(name, "\n")] = 0;

                // Baca status

                snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);

```

```

FILE *status_file = fopen(path, "r");

char state[20] = "Unknown";

if(status_file) {

    char status_line[100];

    while(fgets(status_line, sizeof(status_line), status_file)) {

        if(strstr(status_line, "State:")) {

            sscanf(status_line, "State:\t%s", state);

            break;

        }

    }

    fclose(status_file);

}

printf("%-8s %-20s %s\n", entry->d_name, name, state);

count++;

}

fclose(file);

}

}

}

closedir(dir);

printf("\n");

}

```

```

// Fungsi untuk membaca network interfaces

void get_network_info() {

    printf("NETWORK INTERFACES\n");

    print_line(50);


    FILE *file = fopen("/proc/net/dev", "r");

    if(!file) {

        printf("Error: Cannot open /proc/net/dev\n");

        return;

    }


    char line[256];

    int line_count = 0;


    printf("%-12s %-12s %s\n", "INTERFACE", "RX Bytes", "TX Bytes");

    print_line(40);


    while(fgets(line, sizeof(line), file)) {

        if(line_count++ < 2) continue; // Lewati 2 baris header


        char interface[20];

        unsigned long rx_bytes, tx_bytes;


        if(sscanf(line, "%[^:]: %lu %*u %*u %*u %*u %*u %*u %*u %*u",

```



```

        interface, &rx_bytes, &tx_bytes) == 3) {

// Hapus spasi di nama interface

char *iface = interface;

while(*iface == ' ') iface++;


printf("%-12s %-12lu %-12lu\n", iface, rx_bytes, tx_bytes);

    }

}

fclose(file);

printf("\n");

}

```

// Fungsi untuk load average (NIM genap)

```

void get_load_avg() {

    printf("LOAD AVERAGE\n");

    print_line(50);


    FILE *file = fopen("/proc/loadavg", "r");

    if(!file) {

        printf("Error: Cannot open /proc/loadavg\n");

        return;

    }

```

```

    char line[100];

```

```

if(fgets(line, sizeof(line), file)) {

    float load1, load5, load15;

    sscanf(line, "%f %f %f", &load1, &load5, &load15);

    printf("1 minute : %.2f\n", load1);

    printf("5 minutes : %.2f\n", load5);

    printf("15 minutes: %.2f\n", load15);

}

fclose(file);

printf("\n");

}

int main() {

    clear_screen();

    printf("=== SYSTEM INFORMATION TOOL ===\n\n");


    get_cpu_info();

    get_memory_info();

    get_disk_info();

    get_process_info();

    get_network_info();

    get_load_avg(); // Untuk NIM genap

    return 0;

}

```

BAB IV

Penutup

4.1 Kesimpulan

Berdasarkan pelaksanaan dan analisis terhadap keempat tugas yang telah diselesaikan, dapat disimpulkan bahwa seluruh program berhasil diimplementasikan sesuai dengan persyaratan yang ditetapkan. Program Process Manager mendemonstrasikan kemampuan dalam membuat, mengelola, dan mengoordinasikan proses *child* dan *parent* menggunakan *system call* *fork()*, *waitpid()*, dan *pipe()*, serta menangani sinyal dengan baik. Program File Monitor membuktikan keefektifan *inotify* API untuk memantau perubahan sistem file secara *real-time* dengan filter yang selektif. Custom Shell yang dikembangkan berfungsi sebagai *command-line interpreter* yang tangguh, dengan dukungan *piping*, *redirection*, *background process*, dan *built-in commands*. Terakhir, System Information Tool berhasil mengambil dan menampilkan informasi vital sistem dari */proc* filesystem dengan akurat.

Secara keseluruhan, tugas-tugas ini tidak hanya berhasil mereplikasi fungsi-fungsi inti sistem operasi tetapi juga memberikan pemahaman yang mendalam dan praktis tentang mekanisme internal sistem operasi Linux, seperti manajemen proses, sistem file, dan komunikasi antara *user space* dan *kernel*.